

Bihar Skill Intern

Project Summary and Production Documentation

This printable summary documents the Bihar Skill Intern platform at a high level. Full details are available in PROJECT_DOCUMENTATION.md, SYSTEM_ARCHITECTURE.md, API_REFERENCE.md, and DATABASE_SCHEMA.md.

Project Overview

- Static frontend for public pages, student portal, payment pages, reports, certificates, and admin dashboard.
- Node.js/Express backend hosted separately from the frontend.
- MySQL database for students, admins, courses, enrollments, payments, proofs, attendance, certificates, logs, and resets.
- Razorpay integration for registration payment order creation, verification, status, reconciliation, and webhook processing.

Core Objectives

- Enable online internship registration and payment.
- Provide student course learning, progress tracking, activity proof upload, and document generation.
- Provide admin tools for students, courses, payment amount, proof review, notices, exports, analytics, and certificate controls.
- Preserve production student data and maintain compatibility with older database schemas.

Technology Stack

Layer	Technology
Frontend	HTML, CSS, Vanilla JavaScript, api-config.js
Backend	Node.js, Express.js
Database	MySQL via mysql2/promise
Auth	JWT and bcryptjs
Payments	Razorpay SDK and webhooks
Deployment	GitHub Pages frontend, Render backend

Main Pages

- Public: index.html, instagram.html, 404.html.
- Auth: register.html, login.html, forgot-password.html, reset-password.html, admin-login.html.
- Student: courses.html, course-learning.html, daily-proof.html, profile.html, attendance.html, marksheet.html, certificate.html.
- Documents: report.html, internship-report.html, consent-form.html, registration-documents.html, payment-receipt.html.
- Admin: admin-dashboard.html.

Backend Modules

- server.js mounts API routes, health check, CORS, uploads, logging, sanitization, schema startup, and error handling.
- routes/auth.js handles student/admin auth and password reset.
- routes/admin.js handles admin settings, students, proofs, stats, and courses.
- routes/payments.js handles Razorpay registration and student payment flows.
- routes/students.js handles student profile, courses, progress, and proofs.
- routes/courses.js handles public courses, enrollment, and progress.
- routes/certificates.js handles student certificate retrieval.

Database Summary

- Primary tables: students, pending_registrations, admins, courses, student_courses, payments, certificates, attendance, internship_proofs, marks, notifications, audit_logs, password_resets.
- Runtime compatibility tables: uploaded_files, admin_audit_logs, api_request_logs.
- Important production controls: SKIP_SCHEMA_SYNC, schema compatibility checks, parameterized queries, transactions for critical operations.

Payment Flow

- Student submits registration data and creates a pending registration.
- Backend creates a Razorpay registration order.
- Razorpay Checkout collects payment.
- Backend verifies payment signature or processes webhook.
- Payment is marked completed and pending registration can be promoted to student account.

Certificate and Internship Flow

- Students complete course learning and quiz/progress requirements.
- Students upload daily or bulk activity proofs.
- Admin reviews proof submissions; approved proofs can mark attendance.
- Certificate status can be auto-approved, admin-approved, blocked, or early-unlocked.
- Backend certificate download route currently needs full PDF implementation for production-grade certificate generation.

Security Summary

- bcryptjs password hashing.
- JWT bearer authentication.
- Admin/student role checks.
- Active account checks.
- express-validator request validation.
- CORS allowlist.
- Centralized error handling.
- API request logging and admin audit logging.
- Runtime schema compatibility for safer production deployments.

Known Limitations

- Some admin and frontend state still uses localStorage.
- Refund management is not fully implemented.
- Certificate PDF generation from backend is incomplete.
- Admin role/permission UI and login history are missing.
- Cloud storage is recommended for durable uploaded files on production hosting.
- Automated test coverage is limited.

Maintenance Checklist

- Back up production MySQL before backend deployment.
- Verify Render environment variables and health endpoint.
- Test registration, login, payment, course enrollment, admin login, proof review, and certificate access.
- Keep SKIP_SCHEMA_SYNC=true unless a controlled migration is approved.
- Never commit real .env secrets.
- Do not change routes, IDs, API shapes, or payment flow without explicit migration/testing plan.